🔄 Ciclo while en Python

while · validación · ciclos infinitos · break · continue · else · centinela · bandera

Lógica de Programación 1 · Guía del estudiante

Con `for` sabías de antemano cuántas vueltas habría. Pero muchas veces **no lo sabes**: ¿cuántas veces escribirá mal el usuario su contraseña? ¿cuántos meses tardarás en reunir una meta? ¿cuántos productos pasará el cajero? Para eso existe `while`: **repite un bloque mientras una condición sea True**.

1 · El ciclo while

regresiva.py

```
n = int(input("¿Desde qué número? "))  # 1. valor
inicial
while n > 0:  # 2. se revisa antes de cada vuelta
    print(n)
    n -= 1  # 3. el cuerpo cambia n
print("¡Despegue!")  # al salir del ciclo
```

```
¿Desde qué número? 5
5
4
3
2
1
¡Despegue!
```

Las 3 partes de todo while: (1) la variable de la condición tiene un valor **antes** del ciclo; (2) la condición se revisa **antes** de cada vuelta (si es False desde el principio, el cuerpo no se ejecuta ni una vez); (3) algo **dentro** del cuerpo la cambia. Si falta la parte 3 tienes un **ciclo infinito**.

PSeInt	Python	Ojo
`Mientras cond Hacer … FinMientras`	`while cond:`	revisa la condición antes de cada vuelta
`Repetir … Hasta Que cond`	`while True: … if cond: break`	Python no tiene *Repetir*: ciclo "infinito" con una salida
`FinMientras`	(desaparece)	la sangría marca el cuerpo

Usa `for` cuando...	Usa `while` cuando...
sabes cuántas vueltas habrá (la tabla del 7, la nómina de 25 empleados) o recorres una colección (las letras de un texto).	la parada depende de un evento: un dato válido, un centinela, una meta alcanzada, una opción "salir".

2 · Validar datos y alcanzar metas

pin.py

```
pin = input("PIN de 4 dígitos: ")
while len(pin) != 4 or not pin.isdigit():  # mientras
esté MAL
    print("PIN inválido")
    pin = input("PIN de 4 dígitos: ")  # vuelve a
pedirlo
print("PIN registrado")
```

```
PIN de 4 dígitos: 12a4
PIN inválido
PIN de 4 dígitos: 123
PIN inválido
PIN de 4 dígitos: 2026
PIN registrado
```

ahorro.py

```
saldo = float(input("Ahorro inicial: "))
meta = float(input("Meta: "))
meses = 0
while saldo < meta:
    saldo *= 1.05  # rinde 5 % cada mes
    meses += 1  # contador de vueltas
print(f"Meta alcanzada en {meses} meses: {saldo:.2f}")
```

```
Ahorro inicial: 100000
Meta: 150000
Meta alcanzada en 9 meses:
155132.82
```

No sabíamos cuántos meses: el ciclo los cuenta.

3 · Ciclos infinitos

Accidental (un error)

```
i = 1
while i <= 5:
    print("Vuelta", i)
# ¡olvidamos i += 1!
```

`i` siempre vale 1: imprime "Vuelta 1" para siempre. En el computador se detiene con **Ctrl + C**.

Intencional (un patrón)

```
while True:
    cmd = input("Comando (salir
para terminar): ")
    if cmd == "salir":
        break  # la única salida
    print(f"Ejecutando «{cmd}»…")
print("Sesión cerrada")
```

```
Comando (salir para terminar):
abrir
Ejecutando «abrir»…
Comando (salir para terminar):
guardar
Ejecutando «guardar»…
Comando (salir para terminar):
salir
Sesión cerrada
```

Causas comunes de un ciclo infinito accidental: (1) olvidar actualizar la variable de control; (2) actualizarla en la dirección equivocada (`i += 1` con la condición `i > 0`); (3) usar `!=` con un valor que nunca se alcanza exacto (`x += 2` desde 1 nunca vale 10); (4) un `continue` que salta la actualización. **Protégete:** revisa antes del ciclo los datos que lo harían infinito (un pago de 0, una tasa de 0) o pon un límite de vueltas.

4 · break, continue y else

Instrucción	Qué hace
`break`	sale del ciclo **de inmediato**; las vueltas pendientes no se ejecutan.
`continue`	termina **solo esta vuelta** y salta a la siguiente (en un while, vuelve a revisar la condición).
`else` del ciclo	se ejecuta si el ciclo terminó **normalmente**, es decir, **sin** break.

agotados.py — continue

```
for codigo in range(1, 8):
    stock = int(input(f"Stock del producto {codigo}:
"))
    if stock == 0:
        continue  # agotado: salta al siguiente
    print(f"Producto {codigo}: {stock} unidades
listas")
```

```
Stock del producto 1: 5
Producto 1: 5 unidades listas
Stock del producto 2: 0
Stock del producto 3: 3
Producto 3: 3 unidades listas
Stock del producto 4: 0
Stock del producto 5: 0
Stock del producto 6: 8
Producto 6: 8 unidades listas
Stock del producto 7: 1
Producto 7: 1 unidades listas
```

divisor.py — break y else

```
n = int(input("Número (mayor que 1): "))
d = 2
while d * d <= n:  # basta probar hasta la raíz
    if n % d == 0:
        print(f"{n} = {d} × {n // d}: NO es primo")
        break  # encontré un divisor
    d += 1
else:  # solo si terminó SIN break
    print(f"{n} es primo")
```

```
Número (mayor que 1): 91
91 = 7 × 13: NO es primo
```

```
Número (mayor que 1): 97
97 es primo
```

¿Y con 1? `2 * 2 <= 1` es False: no hay vueltas, no hay break... y el else dice que 1 es primo. **Valida los datos antes del ciclo.**

5 · Centinela y bandera

Un **centinela** es un valor acordado que avisa que **ya no hay más datos** ("escribe fin para terminar"). Nunca se procesa como dato. Patrón: **leer antes del ciclo – revisar – procesar – leer al final del cuerpo**.

peaje.py — centinela "fin"

```
total = 0
vehiculos = 0
tarifa = input("Tarifa (fin para terminar): ")  # leer
ANTES
while tarifa != "fin":  # ¿centinela?
    total += int(tarifa)  # procesar
    vehiculos += 1
    tarifa = input("Tarifa (fin para terminar): ")  #
leer al FINAL
print(f"{vehiculos} vehículos · recaudo {total}")
if vehiculos > 0:
    print(f"Promedio por vehículo: {total /
vehiculos:.2f}")
```

```
Tarifa (fin para terminar): 12500
Tarifa (fin para terminar): 18900
Tarifa (fin para terminar): 12500
Tarifa (fin para terminar): fin
3 vehículos · recaudo 43900
Promedio por vehículo: 14633.33
```

Una **bandera** es una variable booleana que indica si algo **ya ocurrió** (encontrado, acceso, hay_error). Es útil cuando el ciclo tiene varias razones para terminar: al salir, la bandera dice **por qué** salió.

clave.py — bandera acceso

```
CLAVE = "lp1-2026"
acceso = False  # bandera
intentos = 0
while not acceso and intentos < 3:
    intento = input("Contraseña: ")
    intentos += 1
    if intento == CLAVE:
        acceso = True  # la bandera cambia
    else:
        print(f"Incorrecta. Te quedan {3 - intentos}
intento(s)")
if acceso:
    print("Bienvenido")
else:
    print("Cuenta bloqueada")
```

```
Contraseña: hola
Incorrecta. Te quedan 2 intento(s)
Contraseña: lp1-2026
Bienvenido
```

```
Contraseña: a
Incorrecta. Te quedan 2 intento(s)
Contraseña: b
Incorrecta. Te quedan 1 intento(s)
Contraseña: c
Incorrecta. Te quedan 0 intento(s)
Cuenta bloqueada
```

	Centinela	Bandera
¿Qué es?	un dato especial que escribe el usuario	una variable booleana del programa
¿Quién decide parar?	el usuario	el programa, cuando ocurre algo
Ejemplos	notas hasta −1 · precios hasta 0 · "fin"	encontrado · acceso · es_primo

6 · Errores frecuentes

Error	Ejemplo	Causa y corrección
SyntaxError	`while n > 0 · while x = 1:`	faltan los dos puntos · para comparar se usa ==.
SyntaxError	`break` dentro de un if sin ciclo	"'break' outside loop": solo va dentro de un ciclo.
Ciclo infinito	`while i <= 3: print(i)`	nada cambia `i` dentro del cuerpo.
Error lógico	validar con `if`	un if pregunta una sola vez: para "hasta que sea válido" se usa `while`.
Error lógico	leer y sumar en la misma vuelta con centinela	el centinela se suma: lee antes del ciclo y al final del cuerpo.
ZeroDivisionError	`total / cantidad`	si el primer dato fue el centinela, cantidad vale 0: revísalo con un if.

7 · Practica sin computador

¿Termina, es infinito o no entra nunca?

Ciclo	Respuesta y por qué
`i = 10 · while i > 0: · i += 1`	
`x = 1 · while x != 10: · x += 2`	
`n = 5 · while n < 0: · n -= 1`	
`saldo = 100 · while saldo > 0: · saldo -= 30`	
`i = 0 · while i < 10: · if i % 2 == 0: continue · i += 1`	

Sé el intérprete: completa la prueba de escritorio

```
saldo = 100000
meses = 0
while saldo < 150000:
    saldo = saldo * 1.10
    meses += 1
print(meses)
```

¿Qué imprime? _____

Vuelta	¿saldo < 150000?	saldo	meses
1			
2			
3			
4			
5			
6			

Resumen para tomarle foto 📸

Validar
`x = leer()` · `while x inválido:` avisar y leer

Centinela
`x = leer()` · `while x != fin:` procesar · leer

Menú
`while True:` … `if op == salir: break`

Saltar un dato
`if malo: continue`

Varias salidas
`while not ok and n < 3:`

else del ciclo
solo si terminó sin `break`

8 · Caja de herramientas para los ejercicios

Quiero…	Patrón o instrucción
contar vueltas hasta una meta	`while saldo < meta:` · `saldo += saldo * tasa / 100` · `meses += 1`
última cifra · quitarla	`n % 10` · `n // 10`
¿es múltiplo de 10000?	`monto % 10000 == 0`
máximo y mínimo	el primer dato válido es a la vez el máximo y el mínimo; luego se compara cada dato nuevo
el 19 % · quitar un %	`valor * 0.19` · `valor * (1 - p / 100)`
2 decimales	`f"{x:.2f}"`
límite de seguridad	`while condicion and vueltas < 600:`

Antes de programar cada ejercicio responde: ¿qué hace que el ciclo termine? ¿qué variable cambia en cada vuelta? ¿hay un centinela, una bandera o una opción de salida? ¿qué datos podrían volverlo infinito? Luego compruébalo en el 🐍 **Laboratorio** de la lección interactiva (se corrige solo) o en IDLE / Thonny.

9 · Ejercicios propuestos

12 ejercicios de menor a mayor dificultad. Bosqueja tu programa en el espacio y pruébalo con los ejemplos y con casos límite: el centinela como primer dato, un dato inválido, el valor exacto de una meta.

🟢 Nivel 1 · Calentamiento — Un while con validación, un contador o un centinela.

Nivel 1 · while de validación

1. Descuento validado

Una tienda permite que el cajero digite el porcentaje de descuento, pero **solo se aceptan valores de 0 a 50**. Pide el porcentaje (real); **mientras** esté fuera de ese rango, muestra `Porcentaje inválido` y vuelve a pedirlo. Cuando sea válido, pide el precio (real) y muestra `Precio final: X` con 2 decimales.
Entrada: uno o más reales y luego un real · **Salida:** los avisos y el precio final
Ejemplo: 60, −5, 20 y luego 100000 → `Porcentaje inválido` (dos veces) · `Precio final: 80000.00`.
💡 **Pista:** el dato se pide una vez antes del ciclo y otra vez **dentro** de él; la condición del `while` describe cuándo el dato está **mal**.

Nivel 1 · while + contador

2. ¿Cuándo termino de pagar?

Le debes dinero a un amigo y le pagas una cuota fija cada mes (sin intereses). El último mes solo pagas **lo que falte**. Pide la deuda y el pago mensual (reales) y muestra cuántos meses tardas, `Meses: M`, y el valor del último pago, `Último pago: X` (2 decimales). Si el pago mensual es 0 o negativo, muestra `Pago inválido` (¡piensa qué le pasaría al ciclo!).

Entrada: dos reales · **Salida:** meses y último pago, o el aviso

Ejemplo: 1000000 y 300000 → `Meses: 4` · `Último pago: 100000.00`.

💡 **Pista:** no sabes de antemano cuántas vueltas habrá: el ciclo sigue mientras quede deuda. En cada vuelta decide cuánto se paga realmente ese mes.

Nivel 1 · while + centinela

3. Caja registradora

Una caja registradora recibe los precios de los productos uno por uno; el cajero escribe **0** cuando termina. Los precios negativos se rechazan con `Precio inválido` y no se cuentan. Al final muestra `Productos: N`, `Subtotal: X`, `IVA: X` (19 %) y `Total: X`, con 2 decimales. Si no se registró ningún producto, muestra `No se registraron productos`.

Entrada: varios reales terminados en 0 · **Salida:** cuatro líneas o el aviso

Ejemplo: 12000, 3500, −200, 8000, 0 → `Precio inválido` · `Productos: 3` · `Subtotal: 23500.00` · `IVA: 4465.00` · `Total: 27965.00`.

💡 **Pista:** el 0 es un **centinela**: avisa que se acabaron los datos, así que nunca debe sumarse. Piensa dónde se lee el siguiente precio para que el centinela se revise antes de procesarlo.

🟡 Nivel 2 · Intermedio — Interés compuesto, juegos con bandera y aritmética con // y %.

Nivel 2 · while + interés compuesto

4. ¿Cuándo se duplica mi inversión?

Inviertes un capital que crece cada año un porcentaje fijo **sobre el saldo acumulado** (interés compuesto). Pide el capital y la tasa anual en % (reales) y muestra cuántos años pasan hasta que el saldo llega **por lo menos al doble** del capital inicial, `Años: A`, y el saldo en ese momento, `Saldo final: X` (2 decimales). Si la tasa es 0 o negativa, muestra `La inversión nunca se duplica`.
Entrada: dos reales · **Salida:** años y saldo final, o el aviso
Ejemplo: 1000000 al 10 % → `Años: 8` · `Saldo final: 2143588.81`.
💡 **Pista:** el ciclo se repite **mientras** la meta no se haya alcanzado; cuenta las vueltas. ¿Qué pasaría con el ciclo si la tasa fuera 0?

Nivel 2 · while + bandera + contador

5. Adivina el número

Juego para dos: el jugador 1 escribe un **número secreto** (1 a 100) y el jugador 2 tiene **máximo 7 intentos** para adivinarlo. Después de cada intento fallido, muestra `Más alto` si el secreto es mayor o `Más bajo` si es menor. Si acierta, muestra `¡Correcto! Lo lograste en N intentos`; si se le acaban los intentos, `Perdiste. El número era S`.
Entrada: un entero (secreto) y luego los intentos (enteros) · **Salida:** las pistas y el resultado
Ejemplo: secreto 42; intentos 50, 25, 42 → `Más bajo` · `Más alto` · `¡Correcto! Lo lograste en 3 intentos`.
💡 **Pista:** el ciclo tiene **dos** razones para terminar: que acierte o que se acaben los intentos. Una variable booleana (bandera) te dice, al salir, cuál de las dos ocurrió.

Nivel 2 · while + // y %

6. Radiografía de un número

Pide un entero positivo y, usando solo operaciones aritméticas (sin convertirlo a texto), muestra: `Dígitos: C` (cuántos tiene), `Suma de dígitos: S`, `Invertido: R` (el número con las cifras al revés, como entero) y si `Es palíndromo` o `No es palíndromo` (se lee igual al derecho y al revés).

Entrada: un entero positivo · **Salida:** cuatro líneas

Ejemplo: 4070 → `Dígitos: 4` · `Suma de dígitos: 11` · `Invertido: 704` · `No es palíndromo`.

💡 **Pista:** con `% 10` obtienes la última cifra y con `// 10` la quitas. Repite mientras queden cifras. Para armar el invertido, cada cifra nueva "empuja" a las anteriores una posición a la izquierda. Guarda una copia del número original antes de destruirlo.

🟠 Nivel 3 · Avanzado — while True con break y continue, menús y ciclos anidados.

Nivel 3 · while + centinela + continue

7. Cajero automático

Simula un cajero. Pide el saldo inicial (real) y luego montos a retirar hasta que el usuario escriba **0**. Un monto es válido solo si es **positivo y múltiplo de 10000**; si no, muestra `Monto inválido`. Cada retiro cobra una **comisión del 1 %** del monto; si el monto más la comisión supera el saldo, muestra `Fondos insuficientes`. Si se aprueba, descuenta ambos y muestra `Retiro aprobado. Saldo: X`. Al terminar muestra `Retiros: N`, `Comisiones: X` y `Saldo final: X` (dinero con 2 decimales).

Entrada: un real y luego enteros terminados en 0 · **Salida:** un mensaje por retiro y el resumen

Ejemplo: saldo 500000; retiros 100000, 25000, 450000, 200000, 0 → `Retiro aprobado. Saldo: 399000.00` · `Monto inválido` · `Fondos insuficientes` · `Retiro aprobado. Saldo: 197000.00` · `Retiros: 2` · `Comisiones: 3000.00` · `Saldo final: 197000.00`.

💡 **Pista:** cuando un monto no sirve no hay nada más que hacer en esa vuelta: existe una instrucción que salta directo a la siguiente. Revisa las reglas en orden: primero si el monto es válido y luego si alcanza el saldo.

Nivel 3 · ciclos anidados + bandera

8. Cazador de primos

Un número es **primo** si es mayor que 1 y solo es divisible entre 1 y sí mismo. Pide dos enteros a y b (a ≤ b) y muestra en una línea todos los primos entre a y b, inclusive: `Primos: …` separados por espacios (o `Primos: ninguno`). Luego muestra `Cantidad: C` y `Suma: S`.

Entrada: dos enteros · **Salida:** tres líneas

Ejemplo: 1 y 20 → `Primos: 2 3 5 7 11 13 17 19` · `Cantidad: 8` · `Suma: 77`.

💡 **Pista:** un ciclo externo recorre los candidatos y otro interno busca divisores. En cuanto aparece un divisor ya sabes la respuesta: no hace falta seguir buscando. ¿Cómo tratas al 0, al 1 y a los negativos?

Nivel 3 · while True + break + match

9. Carrito de compras con menú

Crea un menú que se repita hasta que el usuario elija salir: **1** Agregar producto (pide precio real y cantidad entera), **2** Aplicar cupón (pide el % de descuento; solo se aceptan cupones de 1 a 30 % —si no, `Cupón inválido` — y solo se puede aplicar **uno**: si ya hay uno, muestra `Ya aplicaste un cupón` **sin pedir el porcentaje**), **3** Ver total (muestra `Total: X`: subtotal con el descuento y luego el IVA del 19 %), **4** Salir (muestra `Gracias. Total a pagar: X`). Cualquier otra opción muestra `Opción no válida`. Dinero con 2 decimales.

Entrada: una secuencia de opciones con sus datos · **Salida:** los mensajes de cada opción

Ejemplo: 1 → 20000 × 2; 2 → 10; 3; 4 → `Total: 42840.00` · `Gracias. Total a pagar: 42840.00`.

💡 **Pista:** un ciclo que solo termina cuando el usuario lo decide se escribe como un ciclo "infinito" con una salida explícita. Guarda en variables el subtotal y el porcentaje del cupón; el total se calcula cada vez que se pide, no se acumula.

🔴 Nivel 4 · Reto ★ — Programas completos: estadísticas, protección contra ciclos infinitos y turnos.

Nivel 4 · centinela + validación + máx/mín

★ 10. Cierre de caja con estadísticas

Al cerrar la tienda se digitan las ventas del día (reales) una por una, terminando con **−1**. Una venta negativa distinta de −1 se rechaza con `Venta inválida` (una venta de 0 sí es válida). Al final muestra `Cantidad: N`, `Total: X`, `Promedio: X`, `Máxima: X` y `Mínima: X`. La meta del día es **1000000**: si se alcanzó, muestra `Meta cumplida: bono de X` (2 % del total); si no, `Faltan X para la meta`. Si no hubo ventas válidas, muestra solo `No hubo ventas`. Dinero con 2 decimales.
Entrada: reales terminados en −1 · **Salida:** el resumen
Ejemplo: 250000, −30, 480000, 120000, −1 → `Venta inválida` · `Cantidad: 3` · `Total: 850000.00` · `Promedio: 283333.33` · `Máxima: 480000.00` · `Mínima: 120000.00` · `Faltan 150000.00 para la meta`.
💡 **Pista:** para el máximo y el mínimo necesitas un punto de partida: puede ser la primera venta válida, o una bandera que diga si ya viste alguna. El centinela y las ventas inválidas nunca deben afectar las estadísticas.

Nivel 4 · while + interés + reporte periódico

★ 11. Plan de ahorro con meta

Quieres reunir una meta de dinero. Cada mes, primero el banco paga un **interés mensual** (en %) sobre el saldo que tenías y luego depositas tu **ahorro mensual**. Pide la meta, el ahorro mensual y la tasa mensual en % (reales). Cada vez que se completa un año (cada 12 meses) muestra `Año k: saldo X`. Cuando se alcance la meta, muestra `Meses: M` y `Saldo final: X`. Protege tu programa de un ciclo infinito: si el ahorro mensual es 0 o negativo muestra `Con ese ahorro nunca llegarás a la meta`, y si después de 600 meses no se ha alcanzado, muestra `La meta tardaría más de 50 años`. Dinero con 2 decimales.
Entrada: tres reales · **Salida:** reporte anual y resultado
Ejemplo: meta 3000000, ahorro 200000, tasa 1 % → `Año 1: saldo 2536500.60` · `Meses: 15` · `Saldo final: 3219379.11`.
💡 **Pista:** la condición del `while` puede combinar dos razones para seguir (¿no he llegado? ¿y aún no paso del límite?). Para saber si se completó un año, piensa en el residuo de dividir el número de meses entre 12. Después del ciclo, averigua cuál de las dos razones lo detuvo.

Nivel 4 · while + turnos + validación

★ 12. Juego del 21

En la mesa hay **21 fichas**. Dos jugadores se turnan (empieza el jugador 1) y en cada turno el jugador escribe cuántas fichas toma: **1, 2 o 3**, y nunca más de las que quedan. Si la jugada no cumple, muestra `Jugada inválida` y el **mismo** jugador vuelve a intentarlo. Después de cada jugada válida muestra `Quedan X fichas`.
Quien toma la última ficha pierde: al final muestra `Gana el jugador N`.
Entrada: enteros (las jugadas) · **Salida:** el estado tras cada jugada y el ganador
Ejemplo: jugadas 3, 3, 3, 3, 3, 3, 3 → … `Quedan 0 fichas` · `Gana el jugador 2` (el jugador 1 tomó la última).
💡 **Pista:** necesitas una variable para las fichas y otra para saber de quién es el turno; el turno solo cambia después de una jugada válida. Piensa qué significa, para elegir al ganador, quién tenía el turno cuando las fichas llegaron a 0.